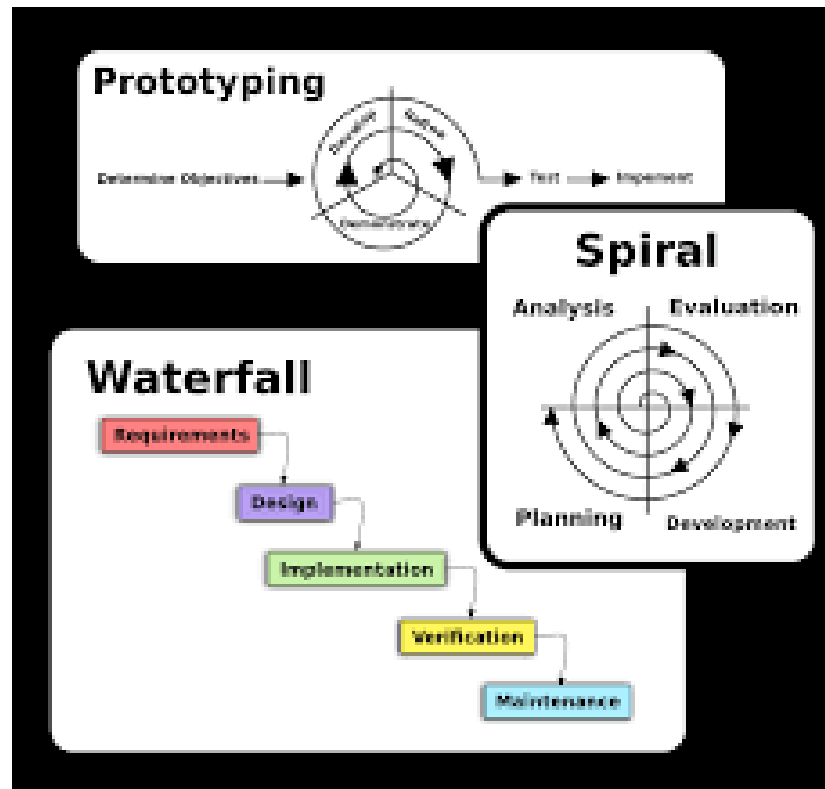
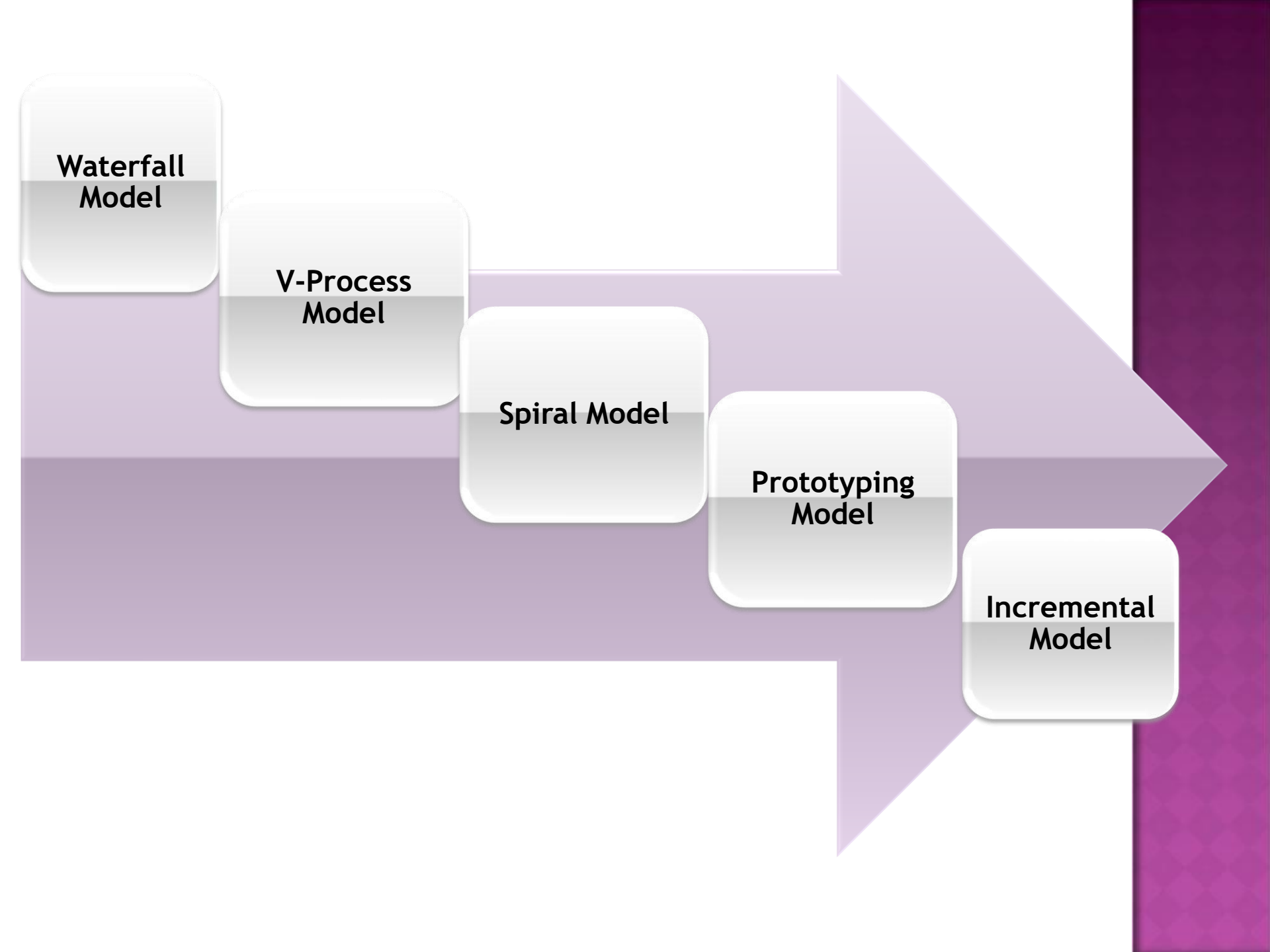


Process Models



By
Farhan M. Shaikh
B.Sc.(CS), M.Sc.(IT), MCA, B.Ed., M.Ed., UGC-NET (Edn.), M.A. (Sociology)
Best Educationist Award (IIE, New Delhi)



**Waterfall
Model**

**V-Process
Model**

Spiral Model

**Prototyping
Model**

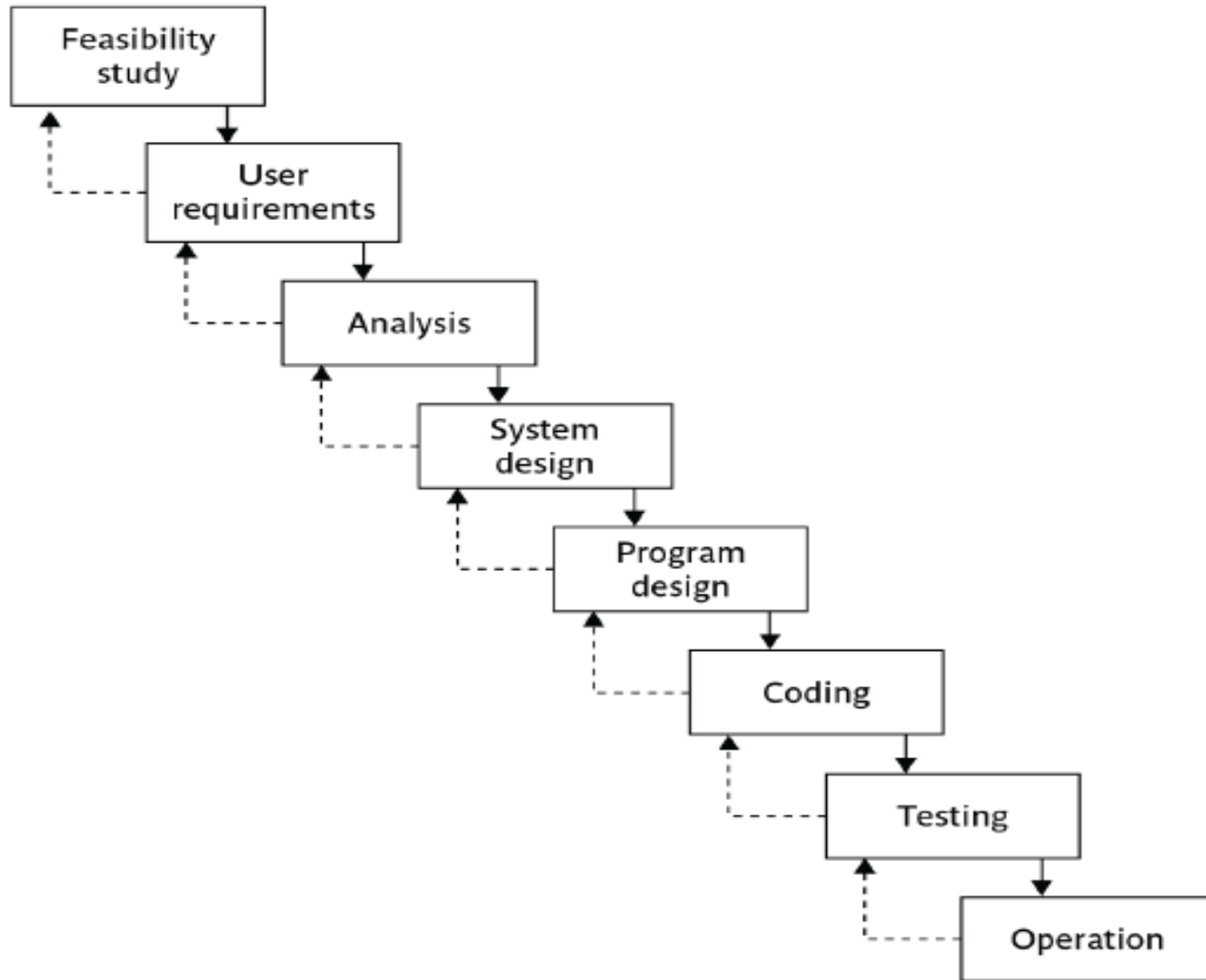
**Incremental
Model**

SOFTWARE PROCESS MODELS

- ◆ Waterfall Model.
- ◆ V-process Model.
- ◆ Spiral Model.
- ◆ Software prototyping.
- ◆ Phased Development Model.
 - ◆ incremental development model.
 - ◆ iterative development model.

Also use of 'agile methods' e.g. extreme programming

WATERFALL MODEL



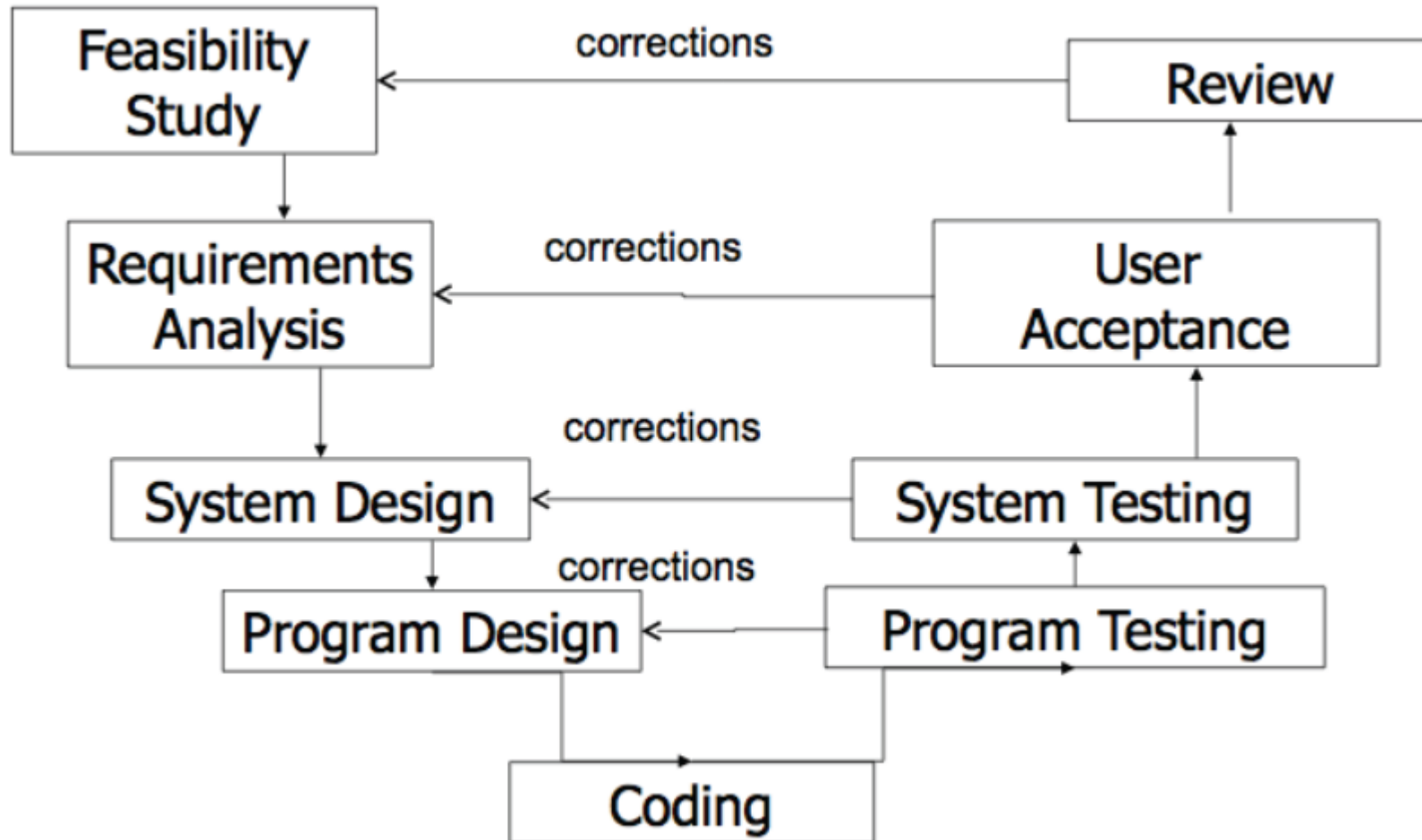
WATERFALL MODEL

- 💧 Classical model of system development.
- 💧 Called one-shot or once-through model.
- 💧 limited scope of iteration. Is this a strength or a limitation??
 - 💧 This is a strength for the WF-model.
 - 💧 Because it is suitable for some projects especially for large projects, we want to avoid reworking tasks that are thought to be completed.
 - 💧 Reworking tasks could result in late delivery.
- 💧 Suitable for systems with well defined requirements.
- 💧 Not suitable for systems of high uncertainty

V-PROCESS MODEL

- ◆ An extension of the waterfall model.
- ◆ V-process model expands the activity box “testing” in the waterfall model.
- ◆ Each step has a matching validation process.
- ◆ Validation process can cause a Loop back to the corresponding stage and reworking the following steps in case of discrepancy.

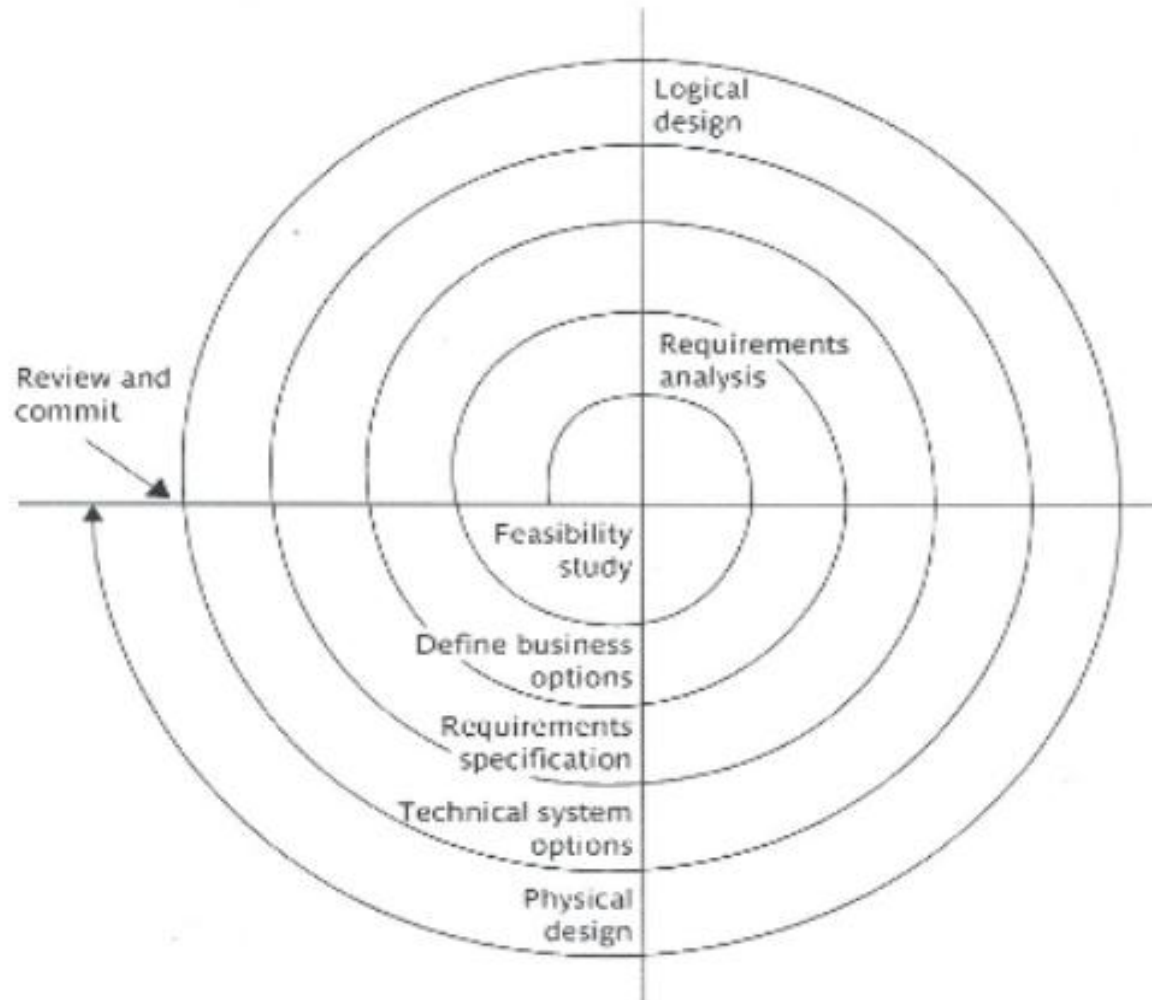
V-PROCESS MODEL



SPIRAL MODEL

- ◆ A greater level of detail is considered at each stage of the project.
- ◆ Represented as a loop or a spiral where the system is considered in more detail.
- ◆ This means **greater confidence** about the probability of success.
- ◆ Each sweep is terminated by an evaluation before the next iteration is embarked upon.

SPIRAL MODEL



SPIRAL MODEL (ADVANTAGES)

- Risk management is easy in this type of model. When you are handling expensive and complex projects, risk management is a must. Moreover, Spiral model has the ability to make any software testing project transparent.
- Customer can see and review the test and different stages
- Projects can be separated into various parts to ease the management difficulty
- Documentation control is strong in this type of methodology
- Project estimate will tend to be more realistic as it progresses.

SPIRAL MODEL (DISADVANTAGES)

- Cannot be used for small projects as it can be expensive
- A vast amount of documentation owing to several intermediate stages
- The end date of the project cannot be calculated at the early stages of the project
- Complex process
- High expertise is required to run the model

PROTOTYPING MODEL

- ◆ **Prototype** is a working model of one or more aspects of the projected system.
- ◆ **Goal**
 - ◆ Gain knowledge
 - ◆ reduce risk and uncertainty
 - ◆ verify a design or implementation approach

The prototype is constructed and tested, quickly and inexpensively to test assumptions.

CLASSIFICATION OF A PROTOTYPE

◆ **Throw-away**

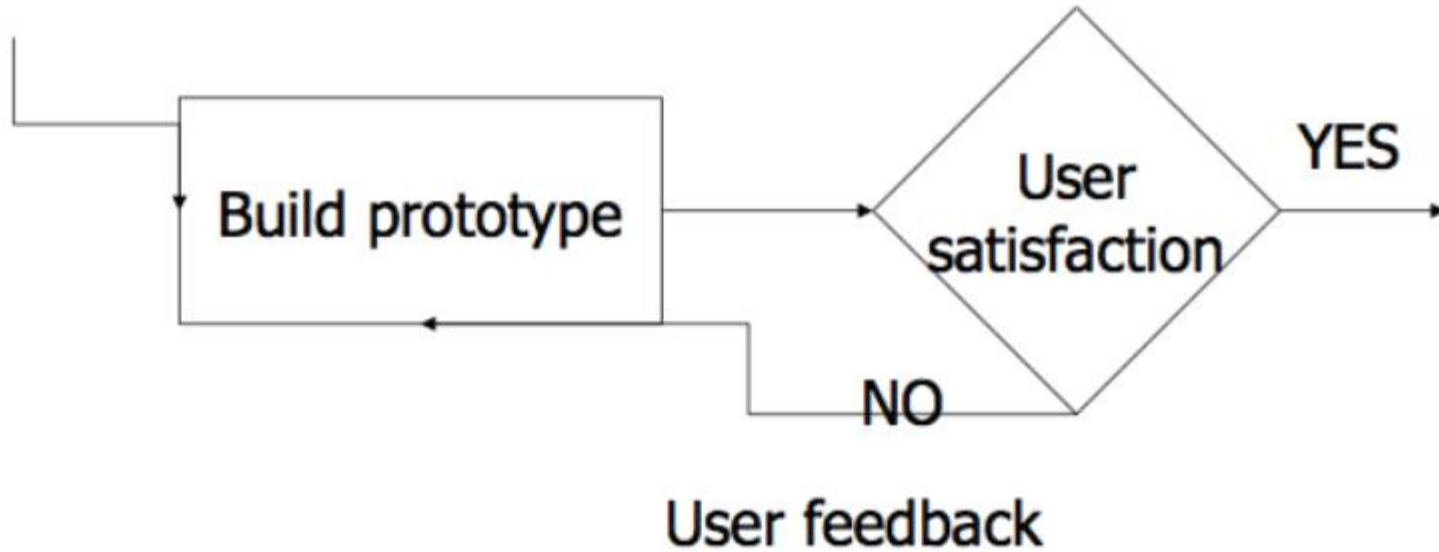
- ◆ Tests out some ideas.
- ◆ Discarded when the true development of the operational system is started.
- ◆ The prototype could be developed using a different SW and HW environment than those that will be used for the final system.

◆ **Example: user interface**

- ◆ **Prototype** :use a desktop application builder to produce an acceptable user interface.
- ◆ **Final system**: use a procedural programming language.

Evolutionary Prototype

PROTOTYPING MODEL



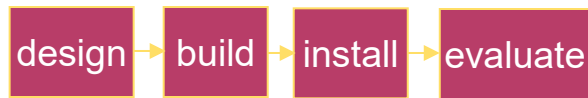
BENEFITS OF PROTOTYPING

- ◆ Learning by doing.
- ◆ Improved communication.
- ◆ Improved user involvement.
- ◆ Clarification of partially-known requirements.
- ◆ Demonstration of the consistency and completeness of a specification
- ◆ Reduced need for documentation.
- ◆ Reduced maintenance costs.
- ◆ Feature constraint.

DRAWBACKS OF PROTOTYPING

- 💧 Users sometimes misunderstand the role of the prototype.
- 💧 Lack of project standards possible.
- 💧 Lack of control.
- 💧 Additional expense.
- 💧 Machine efficiency.
- 💧 Close proximity of developers.

INCREMENTAL DELIVERY



first incremental delivery



second incremental delivery



third incremental delivery



delivered
system

increment
1

increment
2

increment
3

INCREMENTAL DELIVERY

- ◆ Break the system into small components.
- ◆ Implement and deliver small components in sequence.
- ◆ Every delivered component provides extra functionality to the user.

INCREMENTAL APPROACH: BENEFITS

- ◉ feedback from early stages used in developing latter stages
- ◉ shorter development thresholds
- ◉ user gets some benefits earlier
- ◉ project may be put aside temporarily
- ◉ reduces ‘gold-plating’

BUT there are some possible disadvantages

- ◉ loss of economy of scale
- ◉ ‘software breakage’

ITERATION VERSUS INCREMENT

- An iteration represents the state of the overall architecture and the complete deliverable system.
- An increment represents the current work in progress that will be combined with the preceding iteration to form the next iteration.

Incremental



Iterative



SUMMARY - WATERFALL

Strength	Weakness	Types of Projects
Simple Easy to execute Intuitive and logical Easy contractually	All or nothing – too risky Req frozen early May chose outdated hardware/tech Disallows changes No feedback from users Encourages req bloating	Well understood problems, short duration projects, automation of existing manual systems

SUMMARY - PROTOTYPING

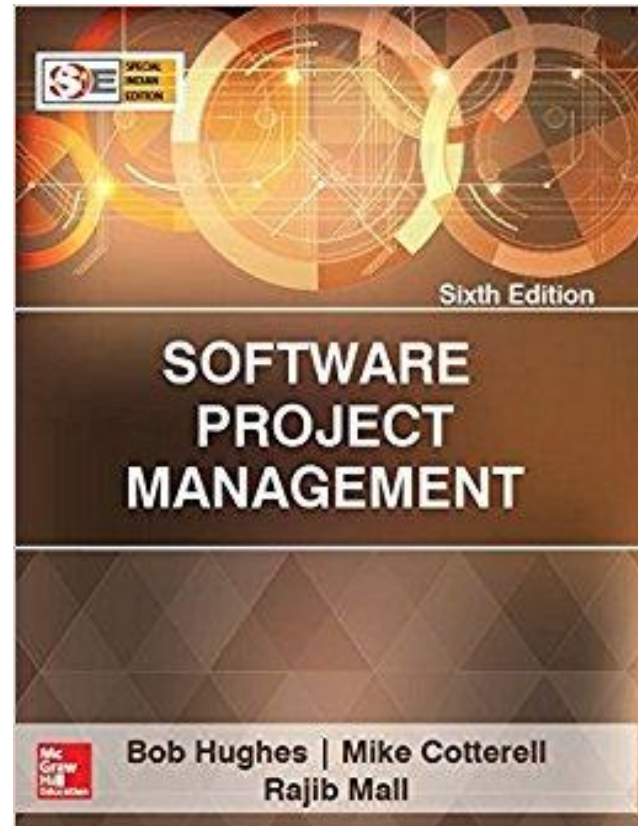
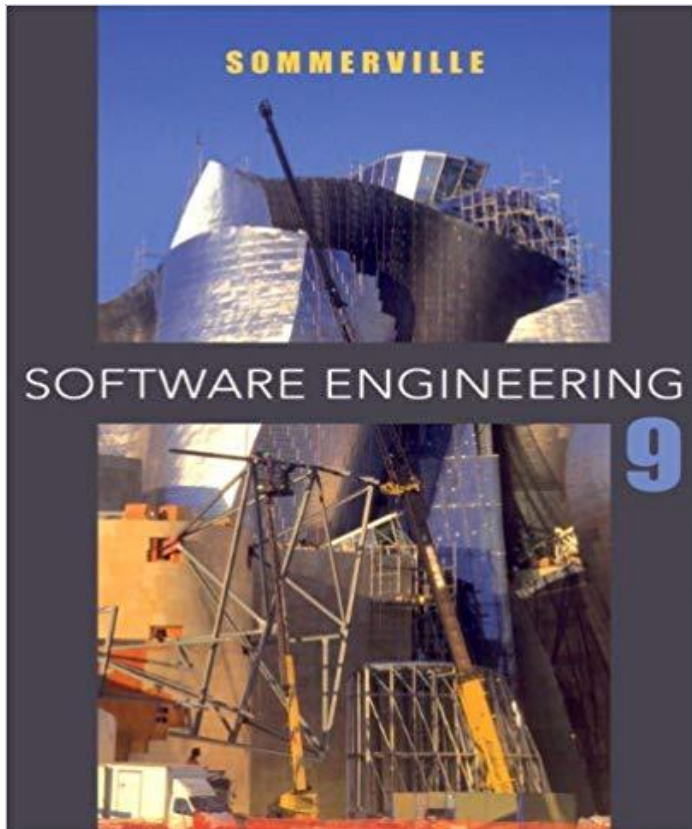
Strength	Weakness	Types of Projects
Helps req elicitation Reduces risk Better and more stable final system	Front heavy Possibly higher cost and schedule Encourages req bloating Disallows later change	Systems with novice users; or areas with req uncertainty. Heavy reporting based systems can benefit from UI proto

SUMMARY - ITERATIVE

Strength	Weakness	Types of Projects
Regular deliveries, leading to biz benefit Can accommodate changes naturally Allows user feedback Avoids req bloating Naturally prioritizes req Allows reasonable exit points Reduces risks	Overhead of planning each iteration Total cost may increase System arch and design may suffer Rework may increase	For businesses where time is imp; risk of long projects cannot be taken; req not known and evolve with time

LITERATURE

- ◉ Software Engineering: Ian Sommerville (9/e)
- ◉ Software Project Management Bob Hughes, Mike Cotterell, Rajib Mall (6/e)



thank
you